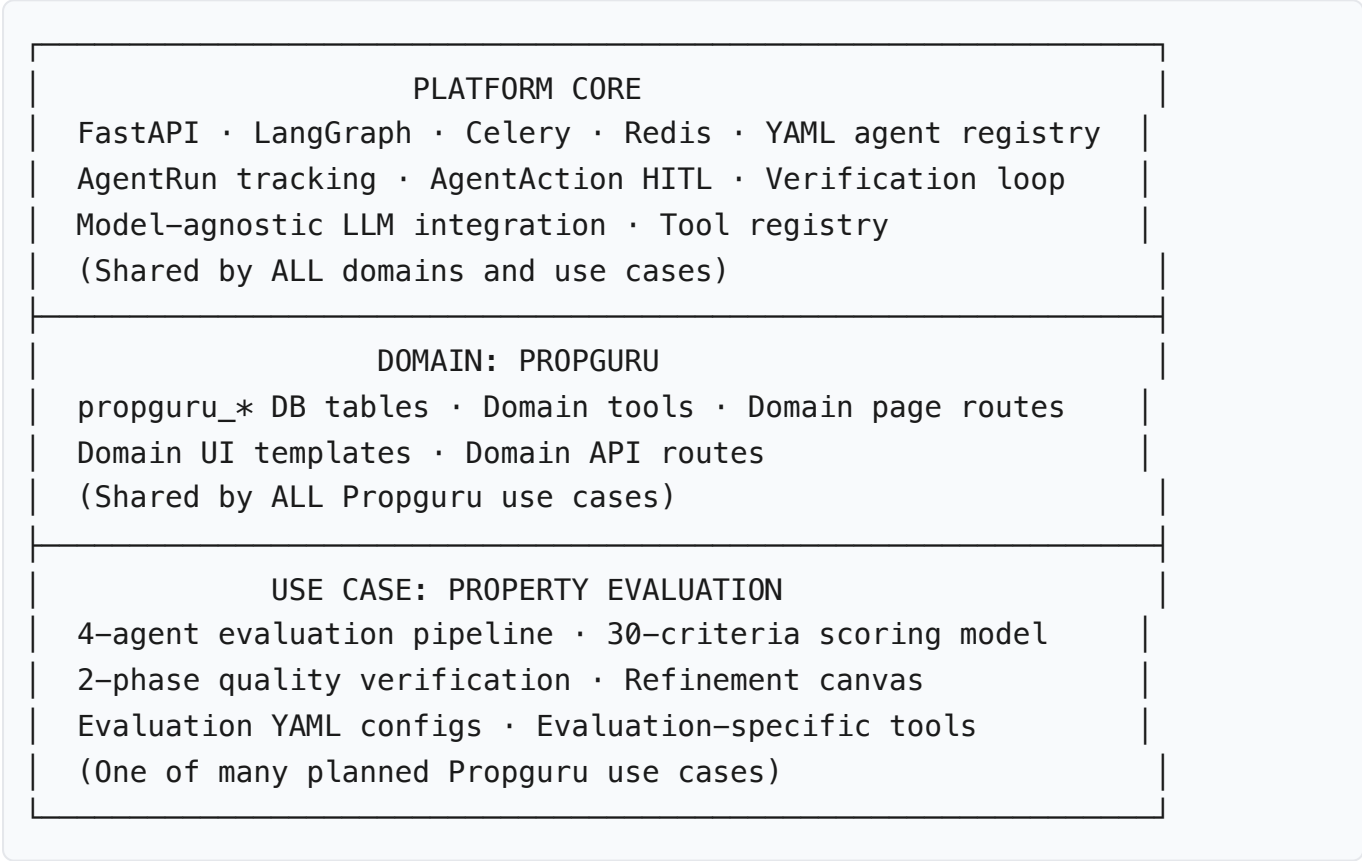
Propguru AI Agent Platform — System Design

> This document shows system architecture of the POC. Actual prod infra system design would be extension of current system design.

---

1. Architecture Philosophy

The system is built in three distinct layers. This separation is the central design decision — it determines how the platform scales across multiple use cases.



Adding a New Use Case

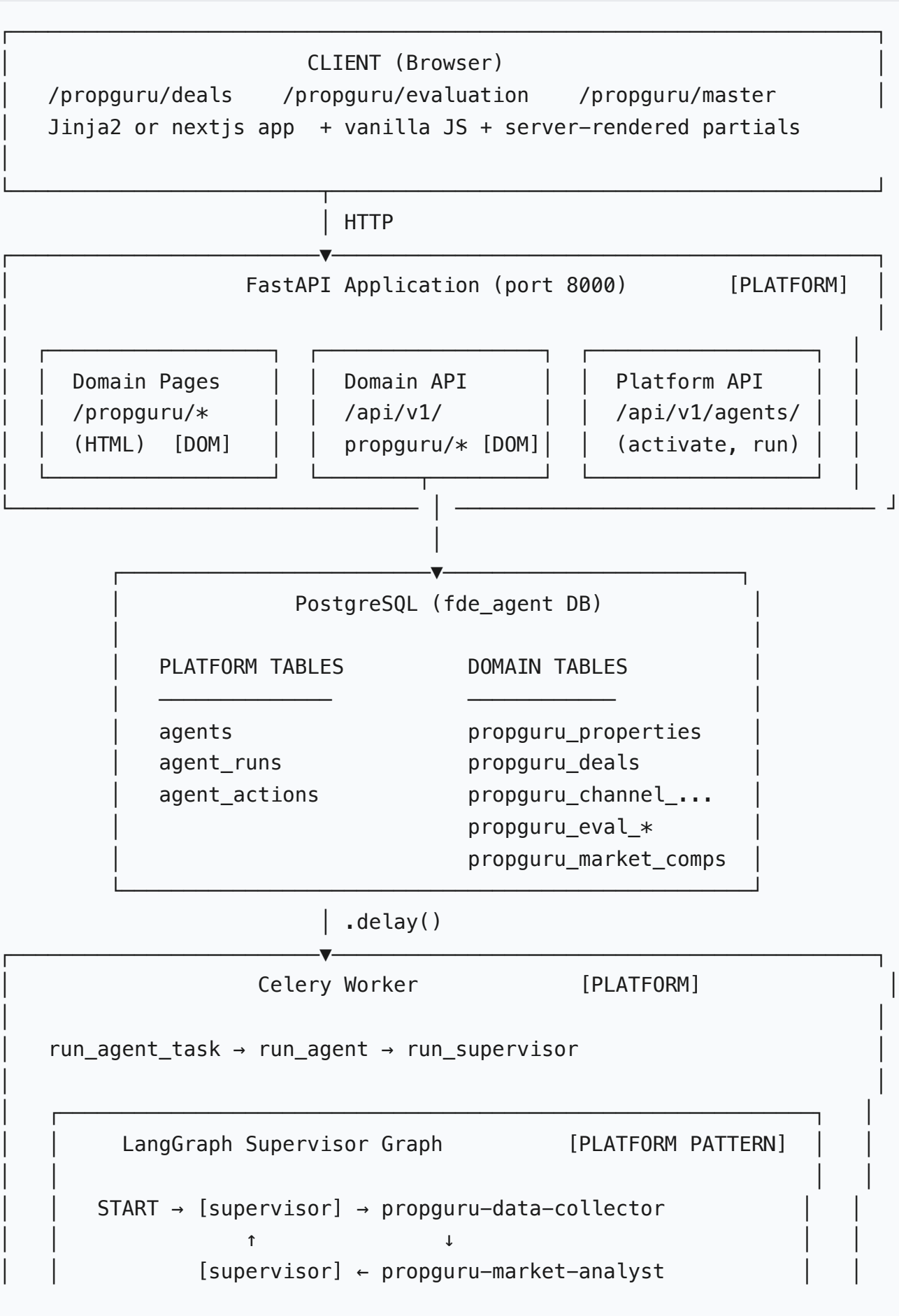
When Propguru adds a new use case (e.g., Channel Partner Lead Scoring, Refurbishment Estimation), the additions are:

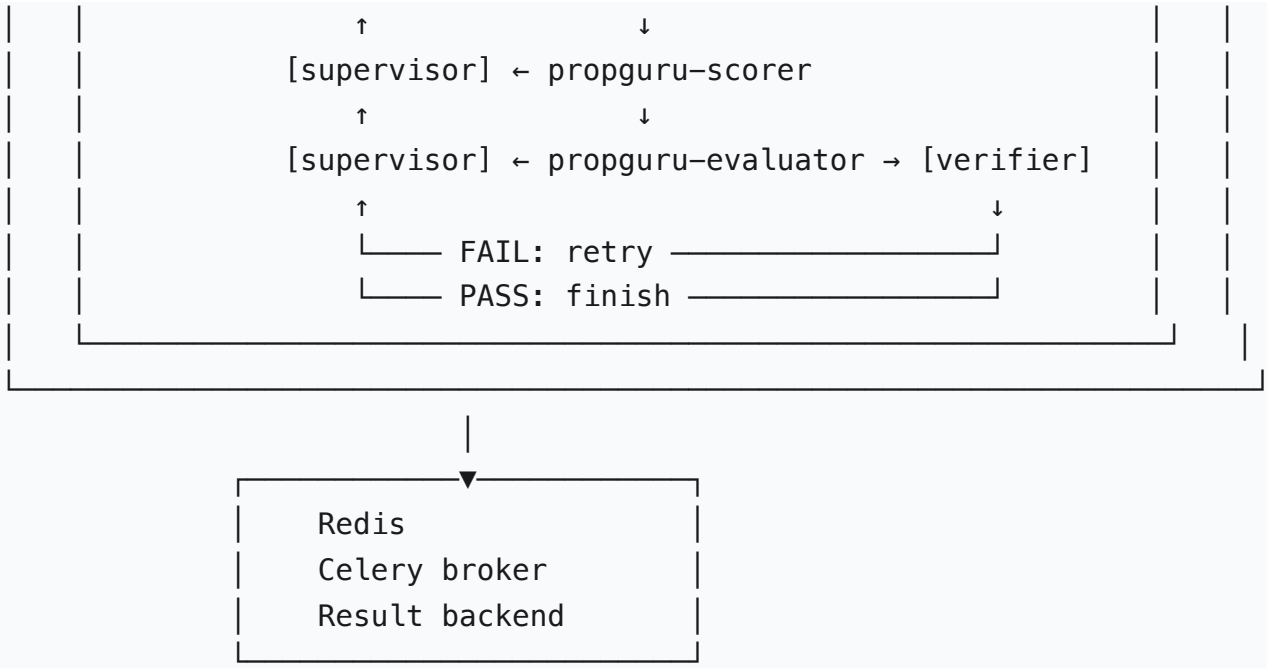
1. **Domain schema** — new DB tables for domain entities (migration only)
2. **Domain tools** — Python functions agents call to read/write domain data
3. **Agent configs** — YAML files defining model, system prompt, tools, feature flags
4. **API routes** — FastAPI routes to trigger runs and expose data
5. **UI templates** — Jinja2 templates or nextjs app for the analyst interface

The platform core changes zero times. Orchestration, HITL, task queue, model routing, audit trail — all reused.

---

2. Full System Architecture




↑ ↓
[supervisor] ← propguru-scorer
↑ ↓
[supervisor] ← propguru-evaluator → [verifier]
↑ ↓
└──── FAIL: retry ────┘
└──── PASS: finish ────┘
Redis
Celery broker
Result backend

Technology Stack

Layer	Technology	Role
Web framework	FastAPI (Python 3.11)	Async REST API + HTML page routes
ORM	SQLAlchemy 2.x (async)	DB access; async in API, sync in Celery
Database	PostgreSQL 16	Persistent store — platform + domain tables
Task broker/backend	Redis 7	Celery message queue + result storage
Task worker	Celery 5	Executes agent pipelines asynchronously
AI orchestration	LangGraph (StateGraph)	Supervisor-worker multi-agent graphs
AI model integration	LangChain + OpenAI SDK	Tool execution and LLM calls
AI models	GPT-4o (workers), GPT-4o-mini (supervisor + grader)	Configurable per agent via YAML
Templates	Jinja2 or nextjs app	Server-rendered pages with partial refreshes
Migrations	Alembic	Incremental schema versioning
Package manager	`uv`	All installs use `uv run`, not `pip`
Container	Docker	`make up` starts all services

---

3. Platform Core

3.1 Agent Registry

Agents are defined in YAML files (`agents/configs/*.yaml`) and stored in the `agents` DB table when activated. No code change is required to change a model, update a prompt, enable a feature flag, or add a new tool to an agent.

```yaml
agent:
  name: propguru-evaluation-supervisor
  type: supervisor            # supervisor | react
  model:
    provider: openai          # openai | anthropic — no code change to
        switch
    name: gpt-4o-mini
    temperature: 0.0
  tools: [...]                # tool names from the tool registry
  feature_flags:
    verification_loop: true
    verification_model_grader_enabled: true
```

`load_agent_config(name)` reads YAML from disk, validates via Pydantic, and returns a typed `AgentConfig`. The Celery task uses this at runtime — no agent state is baked into code.

3.2 Tool Registry

Tools are Python functions registered by name. Agents declare which tools they use in their YAML. The orchestration layer resolves names to callables at runtime.

```python
# Tool registration (src/fde_agent/agent/tools/__init__.py)
TOOL_REGISTRY = {
    "propguru_get_property":         propguru_get_property,
    "propguru_get_criteria":         propguru_get_criteria,
    "propguru_save_evaluation_score": propguru_save_evaluation_score,
    "propguru_calculate_price":      propguru_calculate_price,
    "propguru_propose_evaluation":   propguru_propose_evaluation,
    ...
}
```

Adding a new data source means: write a new tool function + register it + reference it in the agent's YAML. The agent and orchestration layer require no changes.

3.3 Model-Agnostic LLM Integration

The platform builds the LangChain model instance from the `ModelConfig` in the agent's YAML:

```
if config.model.provider == "openai":
    model = ChatOpenAI(model=config.model.name,
        temperature=config.model.temperature)
elif config.model.provider == "anthropic":
    model = ChatAnthropic(model=config.model.name,
        temperature=config.model.temperature)
```

Switching an agent from GPT-4o to Claude Sonnet: change two lines in the YAML and re-activate. Zero code changes.

The OpenAI structured output caveat: `SupervisorDecision` has a `context: dict[str, Any]` field. OpenAI's strict JSON schema mode rejects this. Fix applied:

```
router = model.with_structured_output(SupervisorDecision,
        method="function_calling")
```

This is the only OpenAI-specific line in the platform; it uses function_calling fallback mode which is also supported by Anthropic's tool_use format via LangChain.

3.4 Supervisor-Worker Orchestration (LangGraph)

The platform implements the supervisor-worker pattern as a LangGraph `StateGraph`. This pattern is reused by every multi-agent use case:

```
graph = StateGraph(SupervisorState)

# Supervisor node: reads history, decides next worker or FINISH
graph.add_node("supervisor", supervisor_node)
graph.add_conditional_edges("supervisor", _route_from_supervisor, {...})

# Worker nodes: domain-specific ReAct agents
for worker_name in config.workers:
    graph.add_node(worker_name, worker_node_fn(worker_name, config))
    graph.add_edge(worker_name, "supervisor")   # default: loop back to
        supervisor

# Optional verification node: wired per feature flag
if feature_flags.get("verification_loop"):
    graph.add_node("__verifier__", verifier_node)
    # Override: designated worker routes to verifier instead of supervisor
    graph.add_edge(verification_worker, "__verifier__")
    graph.add_conditional_edges("__verifier__", _verifier_router, {
        "supervisor": "supervisor",
        verification_worker: verification_worker,
    })
```

`SupervisorState` (shared state across all nodes):

```
class SupervisorState(TypedDict):
    messages: Annotated[list, add_messages]
    next_worker: str
    next_instruction: str
    next_context: dict            # carries deal_id, report_id, action_id
        etc.
    rounds: int
    worker_stats: list
    verification_retries: int     # Phase 1 code grader retry count
    grader_flags: list
    grader_verdict: str
    model_grader_retries: int     # Phase 2 model grader retry count
```

3.5 HITL Workflow

The platform HITL pattern is: agent calls a "propose" tool → creates an `AgentAction` record with `status="pending_review"` → analyst sees proposal in the UI → explicitly approves/rejects/refines → only then does any domain state change.

This is enforced at the platform level. No domain use case can bypass it — the "propose" tool only creates a record; the "approve" API endpoint is what actually mutates domain state (deal stage, price, etc.).

3.6 Verification Loop

The verification loop is a platform pattern enabled per agent via feature flags:

```
feature_flags:
  verification_loop: true
  verification_after_worker: "propguru-evaluator"    # which worker
        triggers it
  verification_max_retries: 2
  verification_model_grader_enabled: true
  verification_model_grader_max_retries: 1
  verification_model_grader_model: "gpt-4o-mini"
```

The verifier node is domain-specific (it knows what to check), but the loop wiring — conditional routing, retry counting, state updates, feedback injection — is platform infrastructure.

3.7 Async Task Execution (Celery)

All agent pipeline executions are dispatched as Celery tasks:

```
@celery_app.task(bind=True, max_retries=2, default_retry_delay=10)
def run_agent_task(self, run_id, agent_name, user_message, extra_context):
    # 1. Set AgentRun.status = "running"
    # 2. load_agent_config(agent_name)
    # 3. run_agent/run_supervisor → LangGraph graph.invoke()
    # 4. On success: AgentRun.status = "completed", store output
    # 5. On failure: AgentRun.status = "failed"
    #    If final failure: reset domain entity state (e.g. deal.stage =
        "lead")
    #    Else: raise self.retry(exc=exc)
```

The domain-entity state reset on final failure (`deal.stage = "lead"`) is implemented per use case inside the task's exception handler, using the `deal_id` stored in `run.input["extra_context"]`.

---

4. Database Schema

4.1 Platform Tables

These tables are shared across all domains and use cases.

agents

```
id          UUID PK
name        VARCHAR(100) UNIQUE   -- "propguru-evaluation-supervisor"
description TEXT
version     VARCHAR(20)
config      JSONB                 -- full parsed YAML
is_active   BOOLEAN DEFAULT false
```

agent_runs

One row per pipeline execution.

```
id                  UUID PK
agent_id            UUID FK → agents
status              VARCHAR(20)   -- pending | running | completed |
failed
task_id             VARCHAR(100)  -- Celery task ID
input               JSONB         -- {message, extra_context}
output              JSONB NULLABLE
error               TEXT NULLABLE
input_tokens        INTEGER DEFAULT 0
output_tokens       INTEGER DEFAULT 0
cost_usd            FLOAT DEFAULT 0
langsmith_run_id    VARCHAR(100)
langsmith_trace_url TEXT
otel_trace_id       VARCHAR(64)
started_at          TIMESTAMPTZ
completed_at        TIMESTAMPTZ
created_at          TIMESTAMPTZ
```

agent_actions

HITL proposals. Immutable — never deleted, never overwritten.

```
id              UUID PK
agent_name      VARCHAR(100)
action_type     VARCHAR(50)    -- "propguru_propose_evaluation"
status          VARCHAR(20)    -- pending_review | approved | rejected |
dismissed
title           VARCHAR(300)
summary         TEXT
reasoning       TEXT           -- full AI narrative (also used by model
grader)
input_data      JSONB
output_data     JSONB
approved_by     VARCHAR(100)
approved_at     TIMESTAMPTZ
dismissed_at    TIMESTAMPTZ
notes           TEXT
created_at      TIMESTAMPTZ
updated_at      TIMESTAMPTZ
```

4.2 Domain Tables – Propguru

These tables are shared across all Propguru use cases (evaluation, CP scoring, refurbishment, etc.).

propguru_properties

```
id                  UUID PK
property_code       VARCHAR(20) UNIQUE   -- PROP-001 to PROP-010
address_line1       TEXT
locality            VARCHAR(100)
city                VARCHAR(100)
pincode             VARCHAR(10)
property_type       VARCHAR(30)          -- apartment | independent_house
bedrooms            INTEGER
bathrooms           INTEGER
carpet_area_sqft    FLOAT
built_up_area_sqft  FLOAT
floor_number        INTEGER NULLABLE
total_floors        INTEGER NULLABLE
building_age_years  INTEGER
facing              VARCHAR(20)
latitude            FLOAT
longitude           FLOAT
created_at          TIMESTAMPTZ
updated_at          TIMESTAMPTZ
```

propguru_channel_partners

```
id              UUID PK
cp_code         VARCHAR(20) UNIQUE
name            VARCHAR(200)
cp_type         VARCHAR(30)     -- sourcing | distribution | both
city            VARCHAR(100)
phone           VARCHAR(30)
email           VARCHAR(200)
commission_pct  FLOAT
is_active       BOOLEAN DEFAULT true
created_at      TIMESTAMPTZ
updated_at      TIMESTAMPTZ
```

propguru_deals

```
id                          UUID PK
deal_code                   VARCHAR(20) UNIQUE   -- DEAL-001 to DEAL-005
property_id                 UUID FK → propguru_properties
sourcing_cp_id              UUID FK → propguru_channel_partners (nullable)
sourcing_cp_commission_pct FLOAT
stage                       VARCHAR(30)
  -- lead | evaluation_pending | evaluation_done
  -- | agreement_signed | listed | sold | lost
lead_source                 VARCHAR(50)
notes                       TEXT
target_acquisition_price    FLOAT NULLABLE
final_sale_price            FLOAT NULLABLE
created_at                  TIMESTAMPTZ
updated_at                  TIMESTAMPTZ
```

4.3 Use Case Tables – Evaluation

These tables are specific to the Property Acquisition Evaluation use case.

propguru_evaluation_criteria (30 rows, seeded)

```
id              UUID PK
criterion_code  VARCHAR(20) UNIQUE  -- CRIT-001 to CRIT-030
category        VARCHAR(30)         -- amenity | location | property |
society
name            VARCHAR(200)
description     TEXT
weight          FLOAT               -- 2.0 to 8.0
scoring_type    VARCHAR(20)         -- boolean | scale_1_5 | proximity_km
is_active       BOOLEAN DEFAULT true
sort_order      INTEGER
```

propguru_evaluation_reports

```
id                      UUID PK
deal_id                 UUID FK → propguru_deals
version                 INTEGER DEFAULT 1       -- increments on
refinement
status                  VARCHAR(20)             -- draft | proposed |
approved | rejected
market_rate_per_sqft    FLOAT NULLABLE
base_price              FLOAT NULLABLE
score_factor            FLOAT NULLABLE          -- weighted normalized
score [0,1]
price_premium_pct       FLOAT NULLABLE          -- score_factor × 35%
recommended_price       FLOAT NULLABLE
final_price             FLOAT NULLABLE          -- may differ after
analyst override
confidence              VARCHAR(20) NULLABLE    -- high | medium | low
agent_reasoning         TEXT NULLABLE
analyst_notes           TEXT NULLABLE
approved_by             VARCHAR(100) NULLABLE
approved_at             TIMESTAMPTZ NULLABLE
verification_retries    INTEGER DEFAULT 0       -- Phase 1 grader retries
grader_flags            JSONB NULLABLE
model_grader_retries    INTEGER DEFAULT 0       -- Phase 2 grader retries
created_at              TIMESTAMPTZ
updated_at              TIMESTAMPTZ
```

propguru_evaluation_scores

```
id              UUID PK
report_id       UUID FK → propguru_evaluation_reports
criterion_id    UUID FK → propguru_evaluation_criteria
score           FLOAT                   -- 0–5 (or 0/1 for boolean)
raw_value       TEXT NULLABLE           -- "0.8 km", "yes", "3"
source          VARCHAR(20)             -- "agent" | "analyst"
notes           TEXT NULLABLE
UNIQUE(report_id, criterion_id)
```

Critical business rule: The score save endpoint enforces boolean type constraints:

```python
if scoring_type == "boolean" and score not in (0.0, 1.0):
    raise HTTPException(422, "Boolean criterion score must be 0.0 or 1.0")
```

This prevents AI from storing partial boolean values (e.g. 0.5) that would produce category scores above 100%.

`propguru_market_comps`

```
id                   UUID PK
locality             VARCHAR(100) INDEXED    -- lookup key
avg_price_per_sqft   FLOAT
min_price_per_sqft   FLOAT
max_price_per_sqft   FLOAT
price_trend_6m_pct   FLOAT
transaction_count_6m INTEGER
data_source          VARCHAR(100)
as_of_date           DATE
```

---

5. Evaluation Use Case – Agent Pipeline

5.1 Agent Configs

Six YAML files define the evaluation pipeline agents:

```
agents/configs/
  propguru-evaluation-supervisor.yaml   -- orchestrates 4 workers
  propguru-data-collector.yaml          -- property attribute scoring
  propguru-market-analyst.yaml          -- market comp + base price
  propguru-scorer.yaml                  -- 30-criteria scoring
  propguru-evaluator.yaml               -- price calc + HITL proposal
  propguru-evaluation-refiner.yaml      -- conversational refinement
canvas
```

5.2 Worker Agents Detail

Worker 1: `propguru-data-collector`

Model: GPT-4o | **Tools**: `propguru_create_evaluation_report`, `propguru_get_property`, `propguru_get_criteria`, `propguru_save_evaluation_score`

Applies hard-coded domain rules from property attributes:

Criterion	Scoring Rule
CRIT-021 Floor Level	Ground/1st=2, 2–5th=3, 6–10th=4, 11+=5, top floor−1
CRIT-022 Facing	East=5, North=4, West=3, South=2
CRIT-023 Property Age	0–2y=5, 3–5y=4, 6–10y=3, 11–20y=2, 20+=1
CRIT-025 Power Backup	Boolean from property data
CRIT-026–030 Society	3/5 neutral default when data unavailable

Creates the draft `PropguruEvaluationReport`, returns `report_id`.

Worker 2: `propguru-market-analyst`

Model: GPT-4o | **Tools**: `propguru_get_market_comp`, `propguru_update_report`

1. Fetches comp by exact `locality` match → fallback to broader area → fallback to ₹10,000/sqft + `confidence="low"`
2. `base_price = avg_price_per_sqft × carpet_area_sqft`
3. Writes `market_rate_per_sqft` and `base_price` onto the report

Worker 3: `propguru-scorer`

Model: GPT-4o | **Tools**: `propguru_get_criteria`, `propguru_save_evaluation_score`, `propguru_score_proximity`

Proximity tool converts distance to score:

- < 0.5 km = 5, 0.5–1 km = 4, 1–2 km = 3, 2–3 km = 2, > 3 km = 1

Amenity criteria for independent houses scored 0 (no society amenities).

Worker 4: `propguru-evaluator`

Model: GPT-4o | **Tools**: `propguru_get_evaluation_report`, `propguru_get_evaluation_scores`, `propguru_calculate_price`, `propguru_propose_evaluation`

Calls `propguru_calculate_price` (server-side computation via API), then creates `AgentAction` with `status="pending_review"`.

Price calculation (server-side in `evaluation.py:calculate_price`):

```
def normalize(score, scoring_type):
    if scoring_type == "boolean":      return min(1.0, max(0.0, score))
    if scoring_type == "scale_1_5":   return (score - 1.0) / 4.0
    if scoring_type == "proximity_km": return score / 5.0

score_factor      = Σ(weight × normalize(score)) / Σ(weight)
price_premium_pct = score_factor × 0.35
recommended_price = base_price × (1 + price_premium_pct)

confidence = "high"   if scored_count ≥ 27 and score_factor ≥ 0.60
           = "medium" if scored_count ≥ 21
           = "low"    otherwise
```

This normalization formula must be identical in three places: `evaluation.py:calculate_price`, `pages.py` refine preview partial, and any future reporting logic.

---

6. Evaluation Use Case — Quality Verification

6.1 Phase 1 — Code Grader (Deterministic)

File: `src/fde_agent/agent/propguru_verifier.py`
Cost: Zero (no LLM calls)

Five sequential checks:

Check	Threshold	Flag
Coverage	≥ 28 of 30 criteria scored	`COVERAGE`
Boolean validity	All boolean scores exactly 0.0 or 1.0	`BOOLEAN_INVALID`
Price sanity	`recommended_price` within ±50% of `base_price`	`PRICE_SANITY`
Confidence calibration	`high` requires ≥27 scored AND score_factor ≥ 0.60	`CONFIDENCE_MISMATCH`
Category zero-out	Every category must have ≥1 non-zero score	`CATEGORY_ZERO`

On FAIL: structured feedback injected as `HumanMessage` → evaluator retried (up to 2). On max retries: escalate to analyst inbox with `GRADER_FLAGGED`.

6.2 Phase 2 — Model Grader (LLM-as-judge)

Cost: ~$0.001 per evaluation (GPT-4o-mini, 512 max tokens)

Rubric (weighted average ≥ 6.0/10 to pass):

Criterion	Weight
reasoning_coherence	0.35
price_justification	0.35
market_alignment	0.20
analyst_guidance	0.10

Uses OpenAI SDK directly (not LangChain) to avoid schema compatibility issues with `dict[str, Any]` return types.

Safety: Any infrastructure error → `passed=True` with `MODEL_GRADER_INFRA_ERROR` flag. Reasoning text < 30 chars → skip grader with `MODEL_GRADER_NO_REASONING`. Pipeline never blocked by grader failures.

On FAIL: model grader feedback injected → evaluator retried (up to 1). On max retries: escalate with `REASONING_FLAGGED`.

6.3 Verifier Node Flow

```
__verifier__ called after propguru-evaluator:

  Phase 1: run_propguru_code_grader(report_id)
    FAIL + retries remaining  → dismiss action, reset report, inject
feedback, → evaluator
    FAIL + retries exhausted  → grader_verdict = "escalate" → supervisor
(FINISH)
    PASS                      → continue to Phase 2

  Phase 2: run_propguru_model_grader(report_id, action_id)
    FAIL + retries remaining  → dismiss action, reset report, inject
feedback, → evaluator
    FAIL + retries exhausted  → grader_verdict = "escalate" → supervisor
(FINISH)
    PASS                      → grader_verdict = "pass" → supervisor
(FINISH)
```

---

7. API Design

7.1 Authentication

All endpoints: `X-API-Key: dev-secret-key-change-in-prod` (set via `API_KEY` env var).

7.2 Platform API Routes

```
POST /api/v1/agents/{name}/activate    -- seed YAML config into DB + set
is_active=true
GET  /api/v1/agents                    -- list all registered agents
POST /api/v1/agents/{name}/run         -- trigger a one-off agent run
(queues Celery task)
GET  /api/v1/agents/runs/{run_id}      -- get agent run status and output
```

7.3 Propguru Domain Routes

Master data:

```
GET/POST        /api/v1/propguru/channel-partners
GET/PATCH       /api/v1/propguru/channel-partners/{id}
GET/POST        /api/v1/propguru/properties
GET/PATCH       /api/v1/propguru/properties/{id}
POST            /api/v1/propguru/simulation/seed
POST            /api/v1/propguru/simulation/reset
```

Deal pipeline:

```
GET/POST        /api/v1/propguru/deals
GET             /api/v1/propguru/deals/{id}
PATCH           /api/v1/propguru/deals/{id}/stage
```

7.4 Evaluation Use Case Routes

```
POST   /api/v1/propguru/deals/{id}/evaluate          -- trigger evaluation
pipeline
GET    /api/v1/propguru/deals/{id}/evaluation         -- get latest report
GET    /api/v1/propguru/evaluations/{report_id}       -- full report with
scores
GET    /api/v1/propguru/evaluations/{report_id}/scores  -- scores grouped
by category
POST   /api/v1/propguru/evaluations/{report_id}/calculate-price  --
(called by tool)
POST   /api/v1/propguru/evaluations/{report_id}/grader-result    --
(called by verifier)
PATCH  /api/v1/propguru/evaluations/{report_id}/approve
PATCH  /api/v1/propguru/evaluations/{report_id}/reject
```

Route files:

```
src/fde_agent/api/routes/propguru/
  pages.py       -- HTML page routes (GET /propguru/*)
  deals.py       -- Deal CRUD + evaluation trigger
  evaluation.py  -- Score CRUD, price calculation, grader result,
approve/reject
  master.py      -- Properties, channel partners CRUD + PATCH
  simulation.py  -- /simulation/seed + /simulation/reset
```

---

8. Configuration and Deployment

8.1 Environment Variables

Variable	Purpose
`DATABASE_URL`	Async PostgreSQL (asyncpg — FastAPI)
`DATABASE_URL_SYNC`	Sync PostgreSQL (psycopg2 — Celery worker)
`REDIS_URL`	Redis connection
`CELERY_BROKER_URL`	Redis channel for task queue
`CELERY_RESULT_BACKEND`	Redis for task results
`OPENAI_API_KEY`	Required for all agents using OpenAI provider
`ANTHROPIC_API_KEY`	Required for agents using Anthropic provider
`API_KEY`	Application API key (`X-API-Key` header)
`LANGSMITH_API_KEY`	Optional — LangSmith tracing
`AGENTS_CONFIG_DIR`	Path to `agents/configs/` directory

8.2 \Services

```
services:
  postgres:   # PostgreSQL 16 — persistent volume
  redis:      # Redis 7 — broker + result backend
  api:        # FastAPI (uvicorn) — port 8000, hot-reload via volume mount
  worker:     # Celery worker — does NOT hot-reload; needs restart after
        code changes
  jaeger:     # OpenTelemetry trace UI — port 16686
```
